



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

信号与系统实验大作业报告

课程：信号与系统实验

姓名：郭浩然

学号：2024303980

桌号：18

班级：10 班

学院：民航学院

专业：飞行器控制与信息工程

时间：2025 年 12 月 22 日

目录

1 实验概述与设计目标	3
1.1 实验背景	3
1.2 实验任务要求	3
2 任务一：特定频率噪声抑制系统设计	4
2.1 问题分析	4
2.2 基于零极点配置法的陷波器设计原理	4
2.2.1 数字频率的确定	4
2.2.2 带宽控制的几何解释	4
2.2.3 零点与极点的配置	5
2.3 系统函数推导	5
2.4 幅度频率响应分析	6
2.5 实验结果与分析	6
3 任务四：语音倒放处理	7
3.1 原理分析	7
3.2 实现步骤	7
3.3 实验结果	8
4 任务二与三：音频变调变速算法设计	9
4.1 时频耦合特性	9
4.2 基础方案：重叠相加法 (OLA)	9
4.2.1 时域切片与重组	9
4.2.2 OLA 算法的相位失配问题	10
4.3 时域改进算法：WSOLA	10
4.3.1 核心思想	10
4.3.2 实现机制	10
4.4 频域改进算法：相位声码器 (Phase Vocoder)	12
4.4.1 STFT 与相位处理	12
4.4.2 相位计算	12
4.5 算法对比	13

4.6	实验结果与对比分析	13
4.6.1	场景一：变速不变调（快放）	13
4.6.2	场景二：变调不变速（升调）	14
4.6.3	小结	15
5	实验总结与心得	16
5.1	主要结论	16
5.2	心得与体会	16
5.3	不足与展望	16
A	附录：MATLAB 核心源码	17
A.1	任务一：特定频率噪声抑制	17
A.1.1	主程序 (code1.m)	17
A.1.2	绘图脚本 A (figA.m)	17
A.1.3	绘图脚本 B (figB.m)	17
A.2	任务二、三：变速变调算法	18
A.2.1	WSOLA 算法实现 (wsola.m)	18
A.2.2	Phase Vocoder 核心 (pv_core.m)	22
A.2.3	结果绘图 (figure_draw.m)	24
A.3	任务四：语音倒放	26
A.3.1	倒放实现 (code.m)	26

1 实验概述与设计目标

1.1 实验背景

语音信号处理是数字信号处理领域的重要应用方向。在实际通信与音频处理中，信号常受环境噪声干扰，或需根据场景对语音进行变调、变速处理。本次实验旨在运用《信号与系统》课程中的采样定理、离散时间傅里叶变换（DTFT）、Z 变换及数字滤波器设计原理，解决具体的音频处理问题。

1.2 实验任务要求

本次大作业包含四个主要任务，涵盖从噪声抑制到语音特效处理的过程：

1. **特定频率噪声抑制**：针对叠加了 1800 Hz 单频正弦噪声的语音信号，设计数字滤波器进行滤除，尽可能保留原语音频谱特征。
2. **变调不变速处理**：基于短时傅里叶变换（STFT）或相关算法，实现男声与女声转换，要求音调改变但语速不变。
3. **变速不变调处理**：实现语音的快放与慢放，要求语速改变但音调（基频）不变。
4. **语音倒放处理**：实现音频的时域翻转，并分析其时频特性。

本报告将阐述上述任务的原理、算法设计、实现步骤及结果分析。

2 任务一：特定频率噪声抑制系统设计

2.1 问题分析

本任务需去除叠加在语音信号上的 1800 Hz 正弦噪声 $n(t) = \sin(2\pi \cdot 1800t)$ 。人声频率范围通常在 300 Hz 至 3400 Hz，1800 Hz 位于人声能量集中的频段。若采用普通带阻滤波器，会切除较多有效频率成分，导致语音清晰度下降或音色沉闷。

因此，需要设计一个具有极窄阻带的带阻滤波器（陷波器）。该滤波器仅对 1800 Hz 附近的频率产生较大衰减，对其他频率保持单位增益。

2.2 基于零极点配置法的陷波器设计原理

为了获得较好的陷波效果，采用基于 Z 平面零极点配置的方法设计陷波器。

2.2.1 数字频率的确定

设采样频率为 F_s （实验中由 audioread 获取），噪声频率 $f_0 = 1800$ Hz。则需滤除的数字角频率 ω_0 为：

$$\omega_0 = 2\pi \frac{f_0}{F_s} \quad (1)$$

2.2.2 带宽控制的几何解释

利用 Z 平面的几何矢量法配置零极点。系统的幅频响应 $|H(e^{j\omega})|$ 取决于单位圆上的频率点 $e^{j\omega}$ 到零点距离之积与到极点距离之积的比值：

$$|H(e^{j\omega})| \propto \frac{\prod |e^{j\omega} - z_k|}{\prod |e^{j\omega} - p_k|} \quad (2)$$

当 $\omega = \omega_0$ 时，采样点与零点 z_1 重合，增益为 0，实现陷波。当频率稍有偏离 ($\omega = \omega_0 + \Delta\omega$) 时：

- 若仅有零点，阻带会较宽。
- 引入极点 p_1 后，由于 p_1 与 z_1 非常接近 ($r \approx 1$)，采样点到极点的距离也会迅速增加。分母增加抵消了分子增加，使增益迅速回升至 1 (0 dB)。

因此，极径 r 越接近 1，过渡带越陡峭，频率选择性越好。

2.2.3 零点与极点的配置

滤波器系统函数 $H(z)$ 由零极点决定。

- **零点设计：**为消除频率 ω_0 ，在 Z 平面单位圆角度 $\pm\omega_0$ 处设置共轭零点 z_1, z_2 。

$$z_1 = e^{j\omega_0}, \quad z_2 = e^{-j\omega_0} \quad (3)$$

- **极点设计：**为防止阻带过宽，在零点附近放置共轭极点 p_1, p_2 。极点位于单位圆内以保证稳定，且极径 r 取接近 1 的值（本实验取 $r = 0.99$ ），以获得窄带宽。

$$p_1 = re^{j\omega_0}, \quad p_2 = re^{-j\omega_0} \quad (0 < r < 1) \quad (4)$$

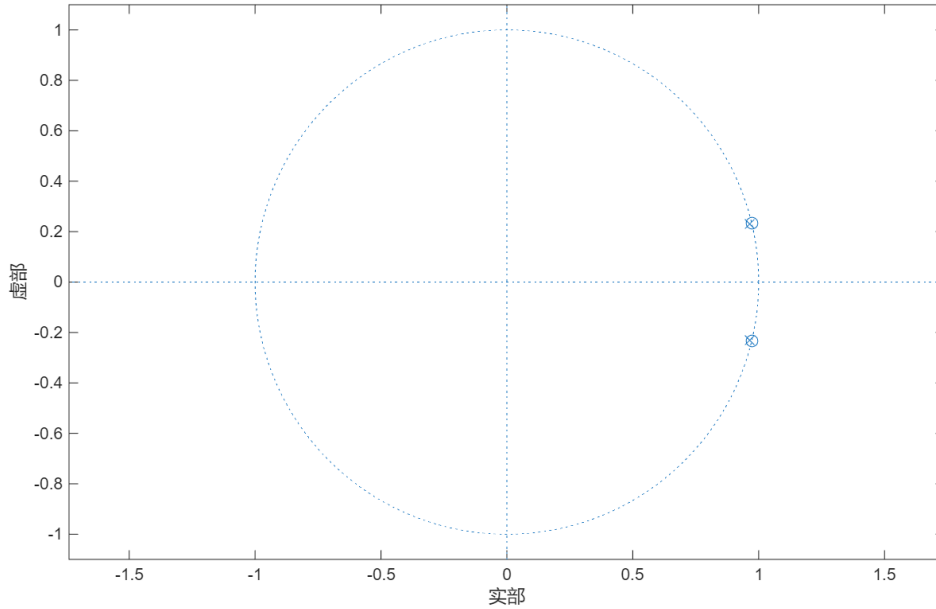


图 1: 设计的 1800Hz 陷波器零极点分布示意图

2.3 系统函数推导

二阶陷波器的系统传递函数 $H(z)$ 表示为：

$$H(z) = b_0 \frac{(z - z_1)(z - z_2)}{(z - p_1)(z - p_2)} \quad (5)$$

其中 b_0 为归一化增益系数，选取使得直流分量（ $\omega = 0$ ）处增益为 1。

展开得：

$$\text{分子} = z^2 - (z_1 + z_2)z + z_1 z_2 = z^2 - 2 \cos(\omega_0)z + 1 \quad (6)$$

$$\text{分母} = z^2 - (p_1 + p_2)z + p_1 p_2 = z^2 - 2r \cos(\omega_0)z + r^2 \quad (7)$$

系统函数 $H(z)$ 标准形式为：

$$H(z) = b_0 \frac{1 - 2 \cos(\omega_0)z^{-1} + z^{-2}}{1 - 2r \cos(\omega_0)z^{-1} + r^2 z^{-2}} \quad (8)$$

2.4 幅度频率响应分析

分析幅频响应 $|H(e^{j\omega})|$ ：当 $\omega = \omega_0$ 时增益极小；当 ω 远离 ω_0 时，增益趋近于 1。

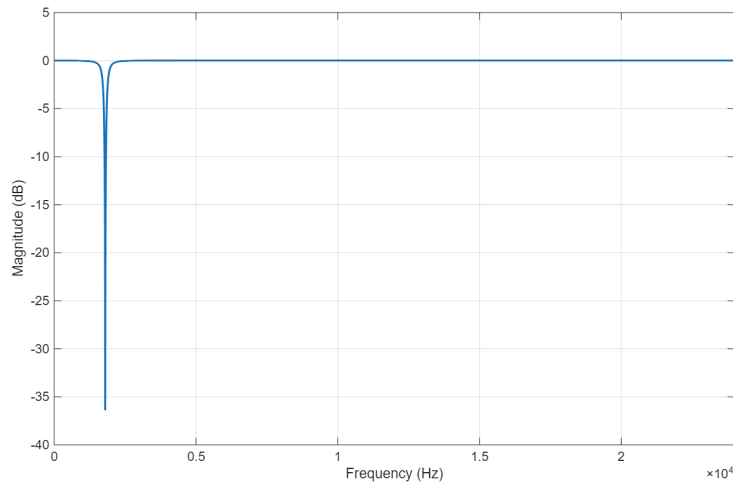


图 2: 设计的陷波器幅频响应特性曲线

2.5 实验结果与分析

对信号进行 FFT 分析，处理前后频谱如图 3 所示。

- 加噪信号：在 1800 Hz 处存在明显尖峰。
- 滤波信号：1800 Hz 处尖峰消失，周围语音频谱成分（1000 Hz 至 3000 Hz）保留较好。

结果表明该陷波器能有效抑制单频噪声。

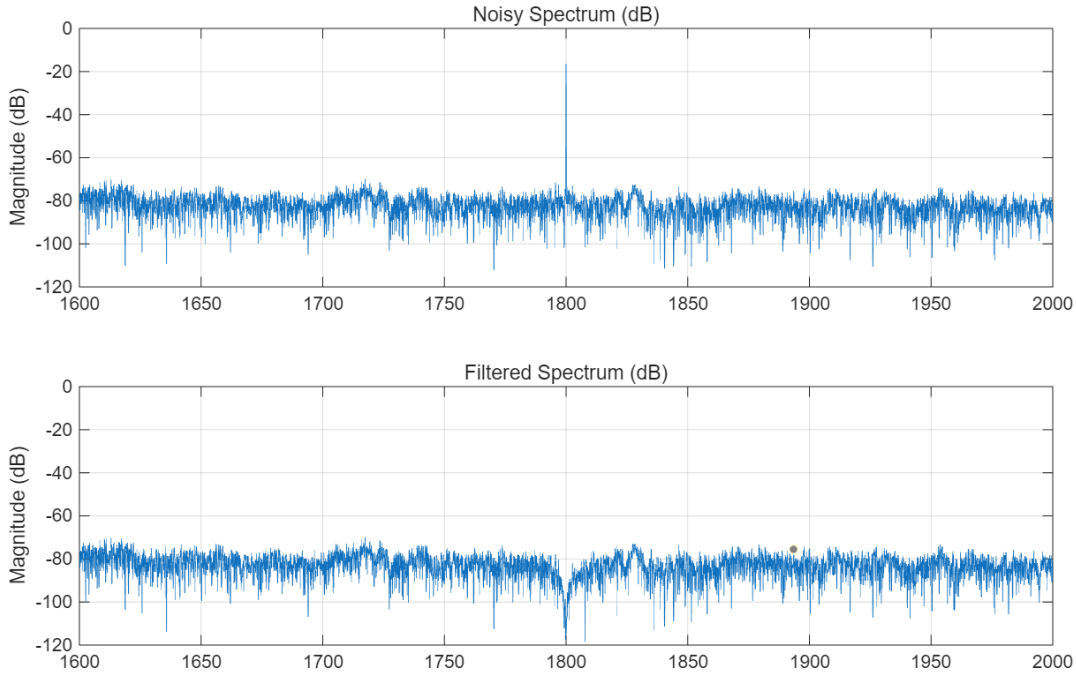


图 3: 噪声抑制前后的频谱对比分析

3 任务四：语音倒放处理

3.1 原理分析

语音倒放是将数字音频信号序列在时间轴上翻转。设原始语音信号为 $x[n]$ ，长度为 N ，倒放后信号 $y[n]$ 为：

$$y[n] = x[N - 1 - n], \quad 0 \leq n \leq N - 1 \quad (9)$$

从频域分析，根据 DTFT 性质，若 $x[n] \leftrightarrow X(e^{j\omega})$ ，则 $x[-n]$ 对应 $X(e^{-j\omega})$ 。对于实数语音信号，频谱具有共轭对称性，即 $|X(e^{j\omega})| = |X(e^{-j\omega})|$ 。

这意味着语音倒放不改变幅度谱，只改变相位谱。听觉上音高和音色基本不变，但语音时序结构（音节顺序、元音辅音位置）颠倒，导致语义无法辨识。

3.2 实现步骤

1. 录制包含数字“1”到“9”的语音信号 $x[n]$ 。2. 利用 MATLAB 函数 `flipud()` 对数组翻转。3. 调用 `audiowrite()` 保存结果。

3.3 实验结果

图 4 为正放与倒放语音的语谱图。可以看出倒放信号的语谱图在时间轴上与原信号互为镜像。例如数字“1”，正向时呈现“起音平稳-收音渐弱”，倒放后变为“起音渐强-突然截断”。这造成了倒放声音中特殊的吸气感和突兀感。

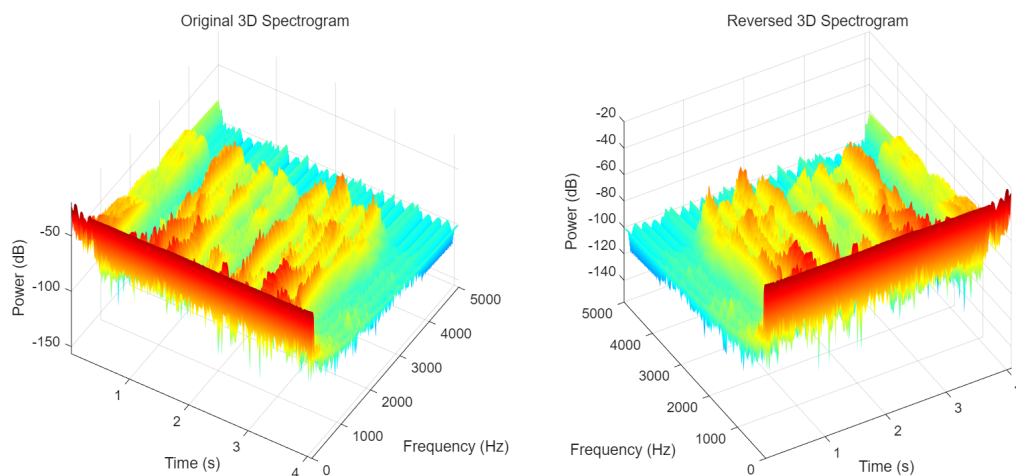


图 4: 正放与倒放语音的语谱图对比

4 任务二与三：音频变调变速算法设计

4.1 时频耦合特性

改变音频播放速度通常会导致音调改变，这源于傅里叶变换的尺度缩放性质。若将时间轴压缩为 $1/a$ ，其频谱扩展 a 倍。

$$x(at) \leftrightarrow \frac{1}{|a|} X\left(\frac{\omega}{a}\right) \quad (10)$$

这种耦合特性使得全局变换无法实现“变调不变速”或“变速不变调”。必须引入短时分析技术解决该问题。变调不变速可通过重采样配合变速不变调实现，因此本节重点讨论变速不变调算法。

4.2 基础方案：重叠相加法 (OLA)

4.2.1 时域切片与重组

重叠相加法(OLA)通过改变分析帧与合成帧的相对密度来实现时间伸缩。如图 5 所示：

- 分析时间轴：以固定步长 H_a 截取数据。
- 合成时间轴：以新步长 H_s 放置数据。

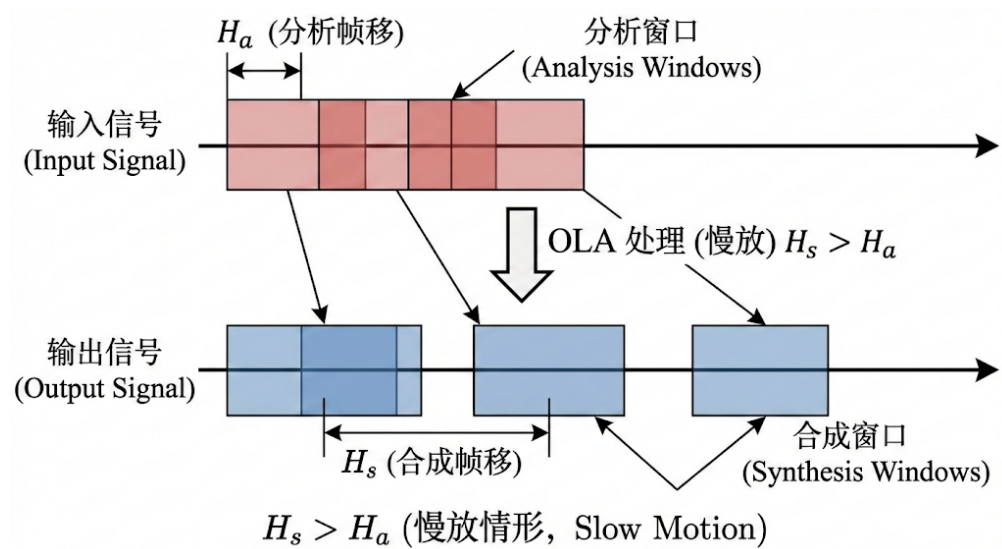


图 5: OLA 算法中分析帧移 H_a 与合成帧移 H_s 的关系示意图

时间伸缩比 $\alpha = H_s/H_a$ 。

4.2.2 OLA 算法的相位失配问题

OLA 虽然改变了时长，但在帧拼接处存在缺陷。强制按照 H_s 步长叠加时，前一帧尾部与后一帧头部往往存在相位差异。如图 6 所示，这种拼接会导致波形跳变，听觉上表现为机械噪音和颤动感，影响语音自然度。

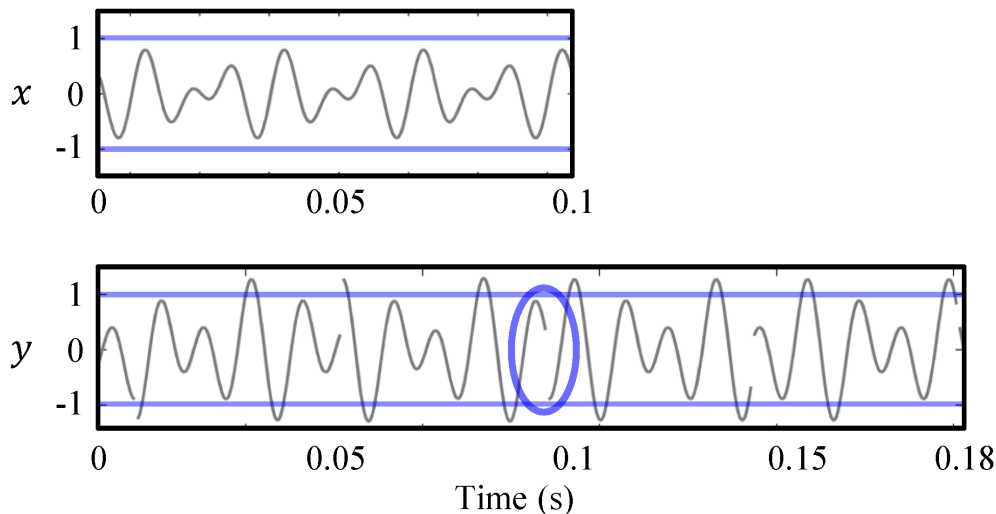


图 6: 传统 OLA 算法导致的波形相位不连续现象

4.3 时域改进算法：WSOLA

4.3.1 核心思想

WSOLA 算法的核心是不直接截取下一帧，而是在目标位置附近寻找最自然的过渡点。

假设已合成前 $k-1$ 帧，第 k 帧理想起始位置为 L_k 。WSOLA 在 L_k 附近的公差窗口内寻找一段与已合成信号尾部最相似的波形作为下一帧。

4.3.2 实现机制

这种“最相似”的寻找过程通过计算归一化互相关函数(Normalized Cross-Correlation, NCC)来实现。其数学本质等价于计算两个波形向量在高维空间中的余弦相似度，旨在消除信号幅度（音量）差异的影响，仅通过波形形状的几何相似性来确定最佳相位对齐点。

设 $y[n]$ 为已合成信号的尾部模板， $x[n]$ 为原信号在搜索窗口内的候选片段，归一化互相关函数 $R(\delta)$ 定义为：

$$R(\delta) = \frac{\sum_{m=0}^{L-1} y[m] \cdot x[L_k + \delta + m]}{\sqrt{\sum_{m=0}^{L-1} (y[m])^2} \cdot \sqrt{\sum_{m=0}^{L-1} (x[L_k + \delta + m])^2}} \quad (11)$$

其中 L 为匹配窗口长度。算法步骤如下：

1. **参考模板**：取已合成信号 y 的末尾波形作为向量 $\vec{A}$ 。
2. **滑动搜索**：在原信号 L_k 附近的公差窗口 $[L_k - \Delta, L_k + \Delta]$ 内滑动截取向量 $\vec{B}(\delta)$ 。
3. **寻找峰值**：计算所有 δ 对应的 $R(\delta)$ ，取最大值对应的偏移量 δ^* ：

$$\delta^* = \underset{\delta \in [-\Delta, \Delta]}{\operatorname{argmax}} R(\delta) \quad (12)$$

如图 7 所示，WSOLA 通过引入最佳偏移量 δ^* ，强制下一帧的起始相位与上一帧的结束相位保持“平行”（即余弦相似度最大），从而实现了平滑拼接。

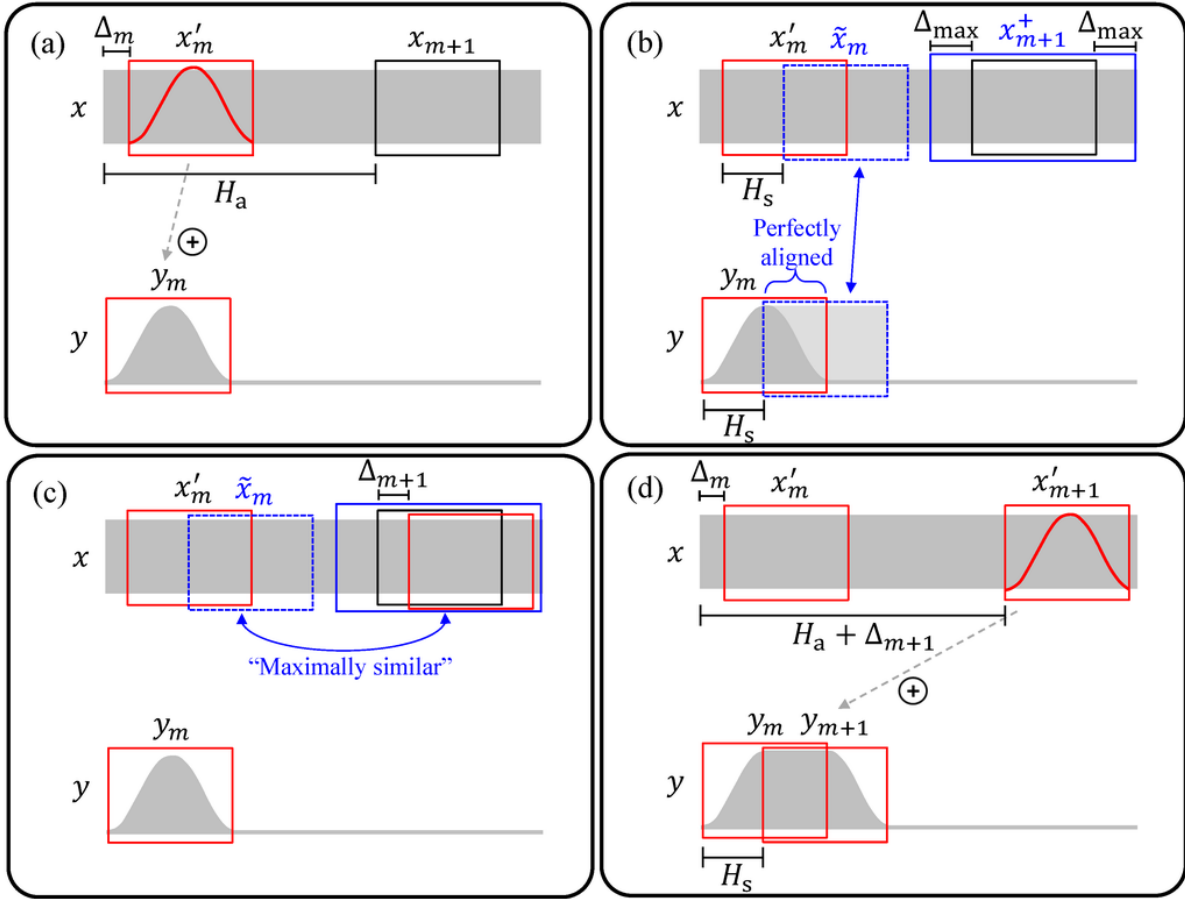


图 7: WSOLA 算法基于互相关搜索的最佳波形匹配机制

4.4 频域改进算法：相位声码器 (Phase Vocoder)

另一种解决相位问题的思路是利用频域方法，直接修正相位谱，即相位声码器。

4.4.1 STFT 与相位处理

Phase Vocoder 利用 STFT 将信号分解。当改变帧跳步时，需计算各频率分量的瞬时频率，并根据新步长重新积分生成相位。

4.4.2 相位计算

步骤如下：

1. 计算相位增量：计算相邻两帧相位差。
2. 计算相位偏差：减去由固有频率产生的理论增量。
3. 相位解卷绕：将相位偏差映射到主值区间，估算真实瞬时频率。
4. 相位重积分：根据合成步长 H_s 累积相位：

$$\phi_s(m) = \phi_s(m-1) + \Delta\phi_{unwrap} \cdot \frac{H_s}{H_a} \quad (13)$$

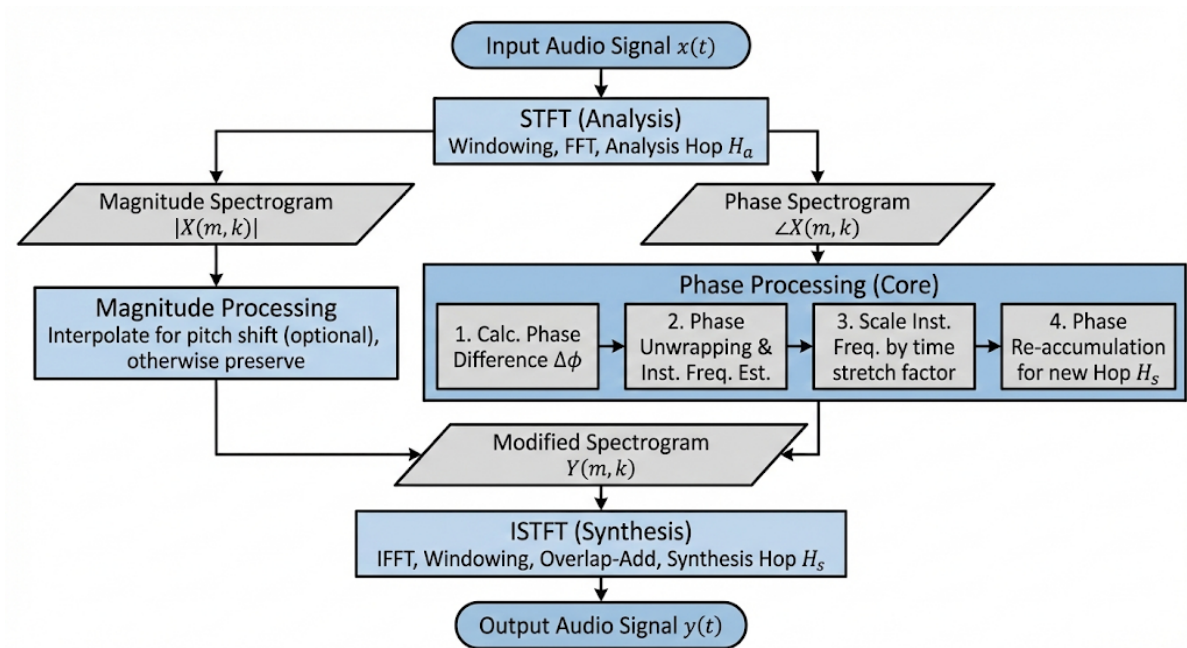


图 8: Phase Vocoder 信号处理流程

4.5 算法对比

表 1: WSOLA 与 Phase Vocoder 算法特性对比

特性	WSOLA (时域)	Phase Vocoder (频域)
原理	波形相似性拼接	瞬时频率估计与相位积分
复杂度	低	高 (需 FFT/IFFT)
瞬态响应	较好	一般 (有模糊感)
相位连续性	依赖匹配精度	较好 (数学连续)
适用场景	语音、打击乐	多声部音乐

4.6 实验结果与对比分析

选取男声语音信号作为输入，分别在“变速不变调”和“变调不变速”场景下对比 WSOLA 和 Phase Vocoder (PV) 算法。

4.6.1 场景一：变速不变调（快放）

1. 实验设置

设定时间伸缩比 $\alpha = 0.7$ （压缩至 70%），保持音调不变。

2. 结果分析

图 9 为处理前后语谱图。

- 时长控制：两算法均成功压缩时长，且语谱图中的水平亮纹位置未偏移，说明音调未变。
- 细节与听感：
 - WSOLA (图 b)：保留了较多纹理细节。听感上清晰度较高，辅音的瞬态特征保留较好，声音较紧实。
 - Phase Vocoder (图 c)：语谱图线条平滑，但听感上有一定的模糊感。由于平滑效应，瞬态冲击力有所削弱。

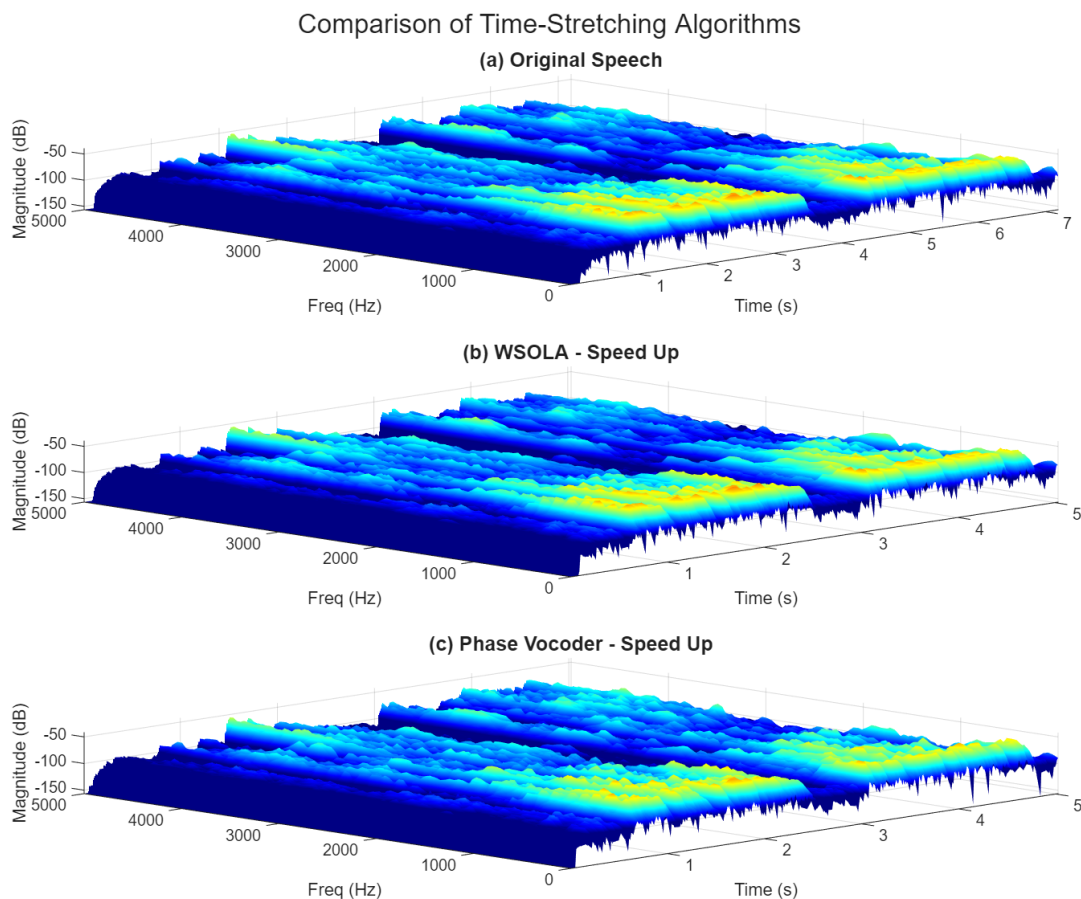


图 9: 变速不变调（快放）实验结果对比：WSOLA vs Phase Vocoder

4.6.2 场景二：变调不变速（升调）

1. 实验设置

升高 4 个半音，时长不变。

2. 结果分析

如图 10 所示。

- 频率位移：图 (b) 和 (c) 能量亮纹整体上移，符合升调预期。
- 音质分析：
 - WSOLA (图 b)：保留了波形内部结构，听感较自然，类似真实改变音高。
 - Phase Vocoder (图 c)：存在不足。虽然水平相位连续，但破坏了频率分量间的垂直相位同步，导致听感上有一定的机械金属感和空间疏离感。

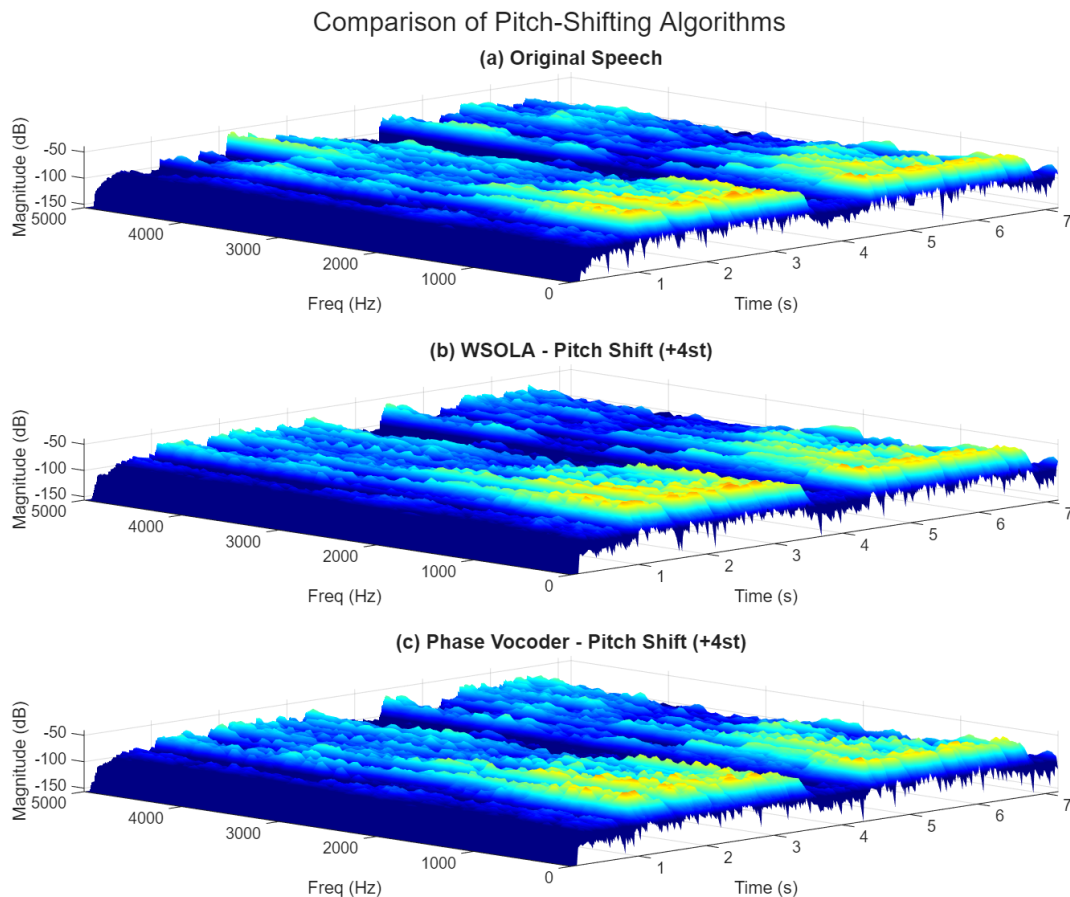


图 10: 变调不变速（升调）实验结果对比：WSOLA vs Phase Vocoder

4.6.3 小结

对比实验表明：

1. **Phase Vocoder** 虽然语谱图视觉平滑，但存在牺牲瞬态清晰度和垂直相位同步的问题，导致听感不够自然。
2. **WSOLA** 通过复用原始波形片段，较好地维持了谐波相位关系和瞬态特征。在单声道语音处理任务中，其听感清晰度和自然度优于 Phase Vocoder。

这说明在语音处理中，保持时域波形的完整性比单纯追求频域相位的连续性更为重要。

5 实验总结与心得

本次《信号与系统》实验大作业涵盖了滤波器设计及语音变速变调算法实现。通过理论推导、MATLAB 仿真及对比分析，加深了对课程知识的理解，并锻炼了工程实践能力。

5.1 主要结论

1. **时频耦合问题**：实验验证了傅里叶变换的尺度缩放性质。简单线性变换无法满足语音处理需求，需引入短时分析或时域拼接技术。
2. **算法性能对比**：实验结果显示，在语音处理方面：
 - **Phase Vocoder** 虽然消除了明显的断裂音，但破坏了信号在垂直方向上的相位同步，导致声音存在机械感。
 - **WSOLA** 虽然原理简单，但通过保留原始波形片段，较好地维持了相位关系。在单声道语音处理中，其效果优于 Phase Vocoder。

这说明算法并非越复杂效果越好，针对具体信号特征选择合适的方法更为关键。

5.2 心得与体会

- **理论到实践的转化**：在设计 1800Hz 陷波器时，通过推导零极点位置并映射到 Z 平面，直观理解了系统函数与频率响应的几何关系，加深了对 Z 变换物理意义的认识。
- **数据分析手段**：学会了使用语谱图分析信号时频特性。通过观察能量纹理，能捕捉到时域波形难以发现的失真细节。

5.3 不足与展望

目前实现的算法在进行大幅度变调（如男变女）时，仅改变了基频，未调整声道共振峰，导致音色听起来像“假声”。未来可尝试引入源-滤波器模型，或基于深度学习的方法，以实现更自然的音色转换。

报告人：郭浩然

日期：2025 年 12 月 22 日

A 附录：MATLAB 核心源码

A.1 任务一：特定频率噪声抑制

A.1.1 主程序 (code1.m)

```

1  [x, fs] = audioread(' ../test1.wav');
2  if size(x, 2) > 1; x = x(:, 1); end
3
4  t = (0:length(x)-1)'/fs;
5  noise = 0.3 * sin(2*pi*1800*t);
6  x_noisy = x + noise;
7
8  f0 = 1800;
9  w0 = 2*pi*f0/fs;
10 r = 0.99;
11
12 b = [1, -2*cos(w0), 1];
13 a = [1, -2*r*cos(w0), r^2];
14 k = sum(a) / sum(b);
15 b = b * k;
16
17 y_clean = filter(b, a, x_noisy);
18 % y_clean(1:10000) = 0;
19
20 disp(length(x_noisy));
21
22 audiowrite('task1_noisy.wav', x_noisy, fs);
23 audiowrite('task1_filtered.wav', y_clean, fs);

```

A.1.2 绘图脚本 A (figA.m)

```

1  figure('Color', 'w');
2  zplane(b, a);
3  title('');
4
5  figure('Color', 'w');
6  [H, F] = freqz(b, a, 4096, fs);
7  plot(F, 20*log10(abs(H)), 'LineWidth', 1.5);
8  grid on;
9  xlim([0, fs/2]);
10 xlabel('Frequency (Hz)');
11 ylabel('Magnitude (dB)');

```

A.1.3 绘图脚本 B (figB.m)

```

1  N = length(x_noisy);
2  f = (0:N-1)*(fs/N);
3  X_noisy_db = 20*log10(abs(fft(x_noisy))/N + eps);

```

```

4 Y_clean_db = 20*log10(abs(fft(y_clean))/N + eps);
5
6 figure('Color', 'w');
7 subplot(2, 1, 1);
8 plot(f, X_noisy_db);
9 xlim([1600, 2000]);
10 ylim([-120, 0]);
11 grid on;
12 title('Noisy Spectrum (dB)');
13 ylabel('Magnitude (dB)');
14
15 subplot(2, 1, 2);
16 plot(f, Y_clean_db);
17 xlim([1600, 2000]);
18 ylim([-120, 0]);
19 grid on;
20 title('Filtered Spectrum (dB)');
21 ylabel('Magnitude (dB)');

```

A.2 任务二、三：变速变调算法

A.2.1 WSOLA 算法实现 (wsola.m)

```

1 function main
2
3     filename = 'origin.wav';
4
5     mode = 1;
6     speed_change = 1/0.7;
7     pitch_semitones = 4;
8
9     enable_denoise = false;
10
11
12     [x, fs] = audioread(filename);
13     if size(x, 1) < size(x, 2), x = x'; end
14
15
16     if enable_denoise
17         disp('正在去除底噪...');
18         x = denoise_process(x, fs);
19
20
21     end
22
23
24     if mode == 1
25
26         y = wsola_process(x, speed_change);
27     elseif mode == 2
28
29         ratio = 2^(pitch_semitones / 12);
30         alpha = 1 / ratio;
31         y_temp = wsola_process(x, alpha);

```

```

32
33
34     old_len = size(y_temp, 1);
35     t_old = 0:old_len-1;
36     t_new = 0 : ratio : old_len-1;
37
38     y = zeros(length(t_new), size(x, 2));
39     for c = 1:size(x, 2)
40         y(:, c) = interp1(t_old, y_temp(:, c), t_new, 'linear', 0);
41     end
42 end
43
44 m = max(abs(y(:)));
45 if m > 1, y = y ./ m; end
46
47 sound(y, fs);
48 audiowrite('processed_output.wav', y, fs);
49 end
50
51 function y = wsola_process(x, alpha)
52     [N, nCh] = size(x);
53     winLen = 1024;
54     synHop = winLen / 2;
55     anaHop = round(synHop * alpha);
56     searchWin = 256;
57
58
59     y = zeros(ceil(N / alpha) + winLen * 2, nCh);
60     win = hanning(winLen, 'periodic');
61
62     synPos = 1;
63     anaPos = 1;
64     lastInputPos = 1;
65
66
67     while true
68         naturalPos = lastInputPos + synHop;
69
70
71         if naturalPos + winLen - 1 > N
72             break;
73         end
74
75
76         if anaPos + winLen + searchWin > N
77
78             searchWin = max(0, N - winLen - anaPos);
79             if searchWin < 10, break; end
80         end
81
82         searchStart = max(1, anaPos - searchWin);
83         searchEnd = min(N - winLen, anaPos + searchWin);
84
85
86         if nCh > 1
87             refBlock = sum(x(naturalPos : naturalPos + winLen - 1, :), 2);
88             searchArea = sum(x(searchStart : searchEnd + winLen - 1, :), 2);

```

```

89     else
90         refBlock = x(naturalPos : naturalPos + winLen - 1);
91         searchArea = x(searchStart : searchEnd + winLen - 1);
92     end
93
94
95     L = length(searchArea);
96     M = length(refBlock);
97     nFFT = 2^nextpow2(L + M);
98     X = fft(searchArea, nFFT);
99     Y = fft(flipud(refBlock), nFFT);
100    c = ifft(X .* Y);
101    c = real(c(M:L));
102
103    [~, lag] = max(c);
104    bestInputPos = searchStart + (lag - 1);
105
106
107    for cIdx = 1:nCh
108        frame = x(bestInputPos : bestInputPos + winLen - 1, cIdx);
109        y(synPos : synPos + winLen - 1, cIdx) = ...
110            y(synPos : synPos + winLen - 1, cIdx) + frame .* win;
111    end
112
113    lastInputPos = bestInputPos;
114    synPos = synPos + synHop;
115    anaPos = anaPos + anaHop;
116 end
117
118
119    validLen = synPos + winLen / 2;
120    if validLen > size(y, 1), validLen = size(y, 1); end
121    y = y(1:validLen, :);
122 end
123
124 function y = denoise_process(x, fs)
125
126
127
128    [N, nCh] = size(x);
129    y = zeros(N, nCh);
130
131
132    winLen = 1024;
133    hop = winLen / 2;
134    win = hanning(winLen, 'periodic');
135
136
137    noise_dur = round(0.25 * fs);
138    if noise_dur > N, noise_dur = N; end
139    noise_segment = x(1:noise_dur, :);
140
141    for c = 1:nCh
142
143
144        n_spec_sum = zeros(winLen, 1);
145        count = 0;

```

```

146     pos = 1;
147     while pos + winLen < noise_dur
148         frame = noise_segment(pos:pos+winLen-1, c) .* win;
149         n_spec_sum = n_spec_sum + abs(fft(frame));
150         pos = pos + hop;
151         count = count + 1;
152     end
153     if count > 0
154         noise_profile = n_spec_sum / count;
155     else
156         noise_profile = abs(fft(noise_segment(1:min(winLen, end), c) .* win));
157     end
158
159     processed_ch = zeros(N + winLen, 1);
160     pos = 1;
161     write_pos = 1;
162
163
164
165     alpha = 2.0;
166     beta = 0.01;
167
168     while pos + winLen < N
169         frame = x(pos:pos+winLen-1, c);
170         frame_win = frame .* win;
171
172         spec = fft(frame_win);
173         mag = abs(spec);
174         phase = angle(spec);
175
176
177
178         new_mag = mag - alpha * noise_profile;
179
180
181         new_mag = max(new_mag, beta * mag);
182
183
184         new_spec = new_mag .* exp(1j * phase);
185
186
187         frame_out = real(ifft(new_spec));
188         processed_ch(write_pos : write_pos + winLen - 1) = ...
189             processed_ch(write_pos : write_pos + winLen - 1) + frame_out;
190
191         pos = pos + hop;
192         write_pos = write_pos + hop;
193     end
194
195     y(:, c) = processed_ch(1:N);
196 end
197
198
199     y = y * (hop/winLen) * 4;
200 end

```

A.2.2 Phase Vocoder 核心 (pv_core.m)

```

1  function pv_core
2
3      filename = 'origin.wav';
4
5      mode = 2;
6      speed_ratio = 1/0.7;
7      pitch_semitones = 4;
8
9
10     [x, fs] = audioread(filename);
11     if size(x, 1) < size(x, 2), x = x'; end
12
13     disp(' 正在处理 Phase Vocoder...');
14
15     if mode == 1
16         stretch_factor = 1 / speed_ratio;
17         y = phase_vocoder(x, stretch_factor);
18
19     else
20         target_pitch_ratio = 2^(pitch_semitones/12);
21
22         old_len = size(x, 1);
23         new_len = floor(old_len / target_pitch_ratio);
24         t_old = (0:old_len-1)';
25         t_new = linspace(0, old_len-1, new_len)';
26
27         x_resampled = zeros(new_len, size(x, 2));
28         for c = 1:size(x, 2)
29             x_resampled(:, c) = interp1(t_old, x(:, c), t_new, 'linear');
30         end
31
32         stretch_factor = target_pitch_ratio;
33
34         y = phase_vocoder(x_resampled, stretch_factor);
35
36         if size(y, 1) > old_len
37             y = y(1:old_len, :);
38         elseif size(y, 1) < old_len
39             y(end+1:old_len, :) = 0;
40         end
41     end
42
43     m = max(abs(y(:)));
44     if m > 1, y = y ./ m * 0.98; end
45
46     disp(' 播放结果...');
47     sound(y, fs);
48     audiowrite('output_pv.wav', y, fs);
49     disp(' 已保存为 output_pv.wav');
50 end
51
52 function y = phase_vocoder(x, stretch_factor)
53
54
55     [N, nCh] = size(x);

```

```

56
57
58     nFFT = 2048;
59     hop_syn = nFFT / 4;
60     hop_ana = floor(hop_syn / stretch_factor);
61
62     window = hanning(nFFT, 'periodic');
63
64     omega = 2 * pi * hop_ana * (0:nFFT/2)' / nFFT;
65
66     output_len = ceil(N * stretch_factor);
67     y = zeros(output_len + nFFT, nCh);
68
69     for c = 1:nCh
70         channel_data = [zeros(nFFT, 1); x(:, c); zeros(nFFT, 1)];
71
72         last_input_phase = zeros(nFFT/2 + 1, 1);
73         phase_acc = zeros(nFFT/2 + 1, 1);
74
75         pin = 1;
76         pout = 1;
77
78         while pin + nFFT < length(channel_data)
79
80             grain = channel_data(pin : pin + nFFT - 1) .* window;
81             spec = fft(grain);
82             mag = abs(spec(1:nFFT/2 + 1));
83             phase = angle(spec(1:nFFT/2 + 1));
84
85             delta_phase = phase - last_input_phase;
86
87             dev_phase = delta_phase - omega;
88
89             wrapped_dev = mod(dev_phase + pi, 2*pi) - pi;
90
91             true_freq_dev = wrapped_dev / hop_ana;
92
93             delta_phase_new = (omega + wrapped_dev) * (hop_syn / hop_ana);
94
95             phase_acc = phase_acc + delta_phase_new;
96
97             last_input_phase = phase;
98
99             spec_new = mag .* exp(1j * phase_acc);
100
101             spec_full = zeros(nFFT, 1);
102             spec_full(1:nFFT/2+1) = spec_new;
103             spec_full(nFFT/2+2:end) = conj(flipud(spec_new(2:end-1)));
104
105             grain_out = real(ifft(spec_full)) .* window;
106
107             idx = pout : pout + nFFT - 1;
108             if idx(end) <= size(y, 1)
109                 y(idx, c) = y(idx, c) + grain_out;
110             end
111
112             pin = pin + hop_ana;

```



```

113         pout = pout + hop_syn;
114     end
115 end
116
117 y = y(nFFT+1 : output_len+nFFT, :);
118 end

```

A.2.3 结果绘图 (figure_draw.m)

```

1  function draw_results()
2
3      file_origin = 'origin.wav';
4
5
6      file_wsola_speed = '变速不变调 wsola.wav';
7      file_pv_speed    = '变速不变调 pv.wav';
8
9
10     file_wsola_pitch = '变调不变速 wsola.wav';
11     file_pv_pitch    = '变调不变速 pv.wav';
12
13
14     close all;
15
16
17     figure('Name', 'Speed Comparison 3D', 'Color', 'w', 'Position', [100, 100, 1000,
18     ↪ 800]);
19
20     [x, fs] = read_mono(file_origin);
21     subplot(3, 1, 1);
22     plot_3d_spec(x, fs, '(a) Original Speech');
23
24
25     if isfile(file_wsola_speed)
26         [y, fs] = read_mono(file_wsola_speed);
27         subplot(3, 1, 2);
28         plot_3d_spec(y, fs, '(b) WSOLA - Speed Up');
29     else
30         subplot(3, 1, 2); title('文件未找到: 变速不变调 wsola.wav');
31     end
32
33
34     if isfile(file_pv_speed)
35         [y, fs] = read_mono(file_pv_speed);
36         subplot(3, 1, 3);
37         plot_3d_spec(y, fs, '(c) Phase Vocoder - Speed Up');
38     else
39         subplot(3, 1, 3); title('文件未找到: 变速不变调 pv.wav');
40     end
41
42     sgtitle('Comparison of Time-Stretching Algorithms', 'FontSize', 16);
43
44

```

```

45     figure('Name', 'Pitch Comparison 3D', 'Color', 'w', 'Position', [150, 150, 1000,
    ↪      800]);
46
47
48     subplot(3, 1, 1);
49     plot_3d_spec(x, fs, '(a) Original Speech');
50
51
52     if isfile(file_wsola_pitch)
53         [y, fs] = read_mono(file_wsola_pitch);
54         subplot(3, 1, 2);
55         plot_3d_spec(y, fs, '(b) WSOLA - Pitch Shift (+4st)');
56     else
57         subplot(3, 1, 2); title(' 文件未找到: 变调不变速 wsola.wav');
58     end
59
60
61     if isfile(file_pv_pitch)
62         [y, fs] = read_mono(file_pv_pitch);
63         subplot(3, 1, 3);
64         plot_3d_spec(y, fs, '(c) Phase Vocoder - Pitch Shift (+4st)');
65     else
66         subplot(3, 1, 3); title(' 文件未找到: 变调不变速 pv.wav');
67     end
68
69     sgtitle('Comparison of Pitch-Shifting Algorithms', 'FontSize', 16);
70
71     disp(' 绘图完成! 请手动旋转图形查看最佳 3D 视角。');
72 end
73
74
75 function [x, fs] = read_mono(filename)
76     [x, fs] = audioread(filename);
77     if size(x, 2) > 1
78         x = mean(x, 2);
79     end
80 end
81
82
83 function plot_3d_spec(signal, fs, fig_title)
84
85     win_len = 1024;
86     overlap = 512;
87     nfft = 1024;
88
89
90     [~, F, T, P] = spectrogram(signal, win_len, overlap, nfft, fs);
91
92
93     P_db = 10 * log10(abs(P) + eps);
94
95
96     surf(T, F, P_db);
97
98
99     shading interp;
100    axis tight;

```

```

101     view(-45, 60);
102     colormap jet;
103
104
105     ylim([0, 5000]);
106     caxis([-100, -20]);
107
108
109     title(fig_title, 'FontSize', 12, 'FontWeight', 'bold');
110     xlabel('Time (s)');
111     ylabel('Freq (Hz)');
112     zlabel('Magnitude (dB)');
113     grid on;
114 end

```

A.3 任务四：语音倒放

A.3.1 倒放实现 (code.m)

```

1  [x_orig, fs] = audioread('../test2.wav');
2  if size(x_orig, 2) > 1; x_orig = x_orig(:, 1); end
3  y_reverse = flipud(x_orig);
4
5  win = 2048;
6  noverlap = 1024;
7  nfft = 2048;
8
9  [~, F_orig, T_orig, P_orig] = spectrogram(x_orig, win, noverlap, nfft, fs);
10 P_orig_db = 10*log10(P_orig + eps);
11
12 [~, F_rev, T_rev, P_rev] = spectrogram(y_reverse, win, noverlap, nfft, fs);
13 P_rev_db = 10*log10(P_rev + eps);
14
15 figure('Color', 'w', 'Position', [100, 100, 1200, 500]);
16
17 subplot(1, 2, 1);
18 surf(T_orig, F_orig, P_orig_db);
19 shading interp;
20 axis tight;
21 view(-30, 45);
22 colormap jet;
23 ylim([0, 5000]);
24 xlabel('Time (s)');
25 ylabel('Frequency (Hz)');
26 zlabel('Power (dB)');
27 title('Original 3D Spectrogram');
28
29 subplot(1, 2, 2);
30 surf(T_rev, F_rev, P_rev_db);
31 shading interp;
32 axis tight;
33 view(-30, 45);
34 colormap jet;
35 ylim([0, 5000]);

```

```
36 xlabel('Time (s)');  
37 ylabel('Frequency (Hz)');  
38 zlabel('Power (dB)');  
39 title('Reversed 3D Spectrogram');
```
